

OpenCode Demo Project Documentation

Task Manager API using OpenCode AI Agent

1. Introduction

This document demonstrates the capabilities of **OpenCode**, an open-source AI coding agent, by building a complete project from scratch.

OpenCode enables developers to automate software development tasks such as code generation, debugging, testing, and refactoring using AI agents.

The objective of this project is to evaluate how OpenCode can assist in real-world development workflows by creating a **Task Manager API**.

2. Objectives

- Understand OpenCode's capabilities in software development
- Demonstrate AI-assisted project creation
- Evaluate agent-based coding workflows
- Build a functional REST API using OpenCode

3. Project Overview

The project is a **Task Manager API** built using Node.js and Express.

Features:

- Create tasks
- Read tasks
- Update tasks
- Delete tasks
- Input validation
- Database integration (MongoDB)
- Unit testing

4. System Requirements

Software Requirements:

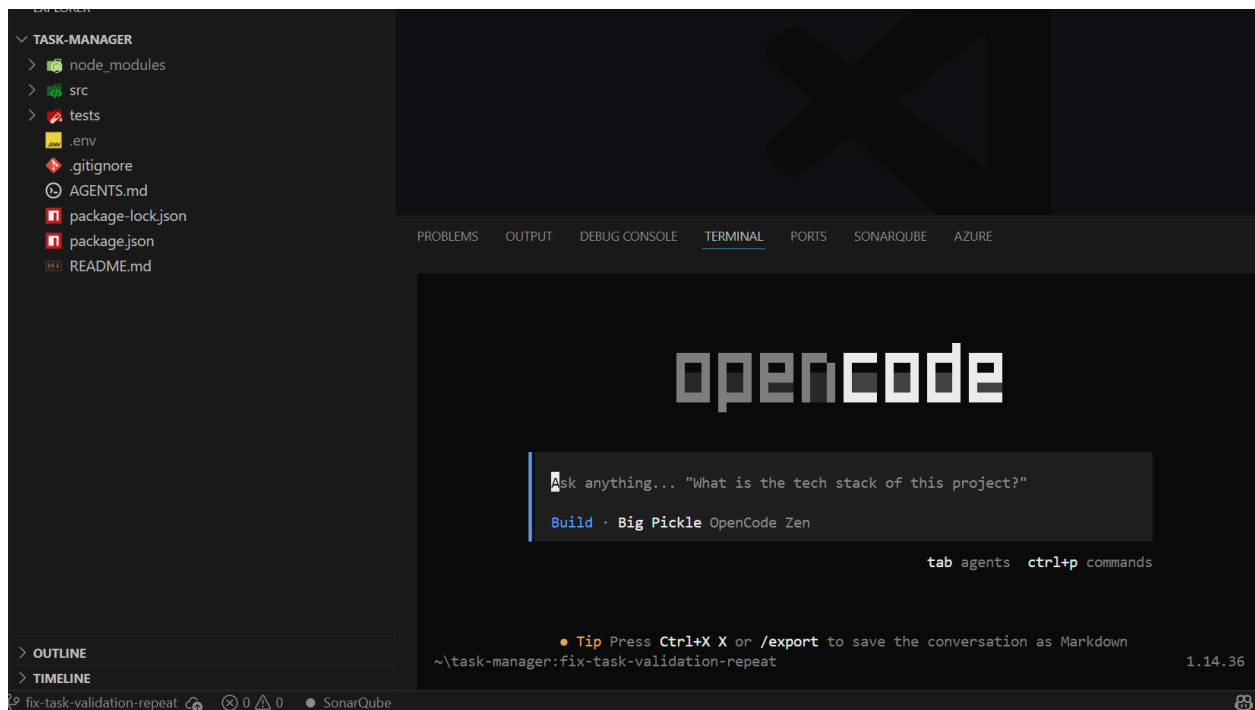
- Node.js (v16 or higher)
- npm (Node Package Manager)
- OpenCode
- MongoDB (local or cloud)

Installation (Windows PowerShell):

```
npm install -g opencode-ai  
opencode
```

5. OpenCode Setup

1. Launch OpenCode: `opencode`
2. Connect AI model: `/connect`
3. Select provider (e.g., OpenAI, Claude)



6. Project Creation Using OpenCode

Step 1: Initialize Project

```
mkdir task-manager  
cd task-manager  
npm init -y
```

Step 2: Generate Application

Instruction given to OpenCode:

Create a Node.js Express API for managing tasks with CRUD operations.
Use a modular folder structure and include validation.

Outcome:

- Project structure generated automatically
- Required dependencies installed
- Initial API implemented

Step 3: Run and Fix Errors

Instruction:

Run the project and fix any errors.

Outcome:

- Dependencies installed
- Server executed
- Errors resolved automatically

Step 4: Add Database Integration

Instruction:

Add MongoDB integration using Mongoose and persist tasks in the database.

Outcome:

- Database connection configured
- Task schema created
- Controllers updated

Step 5: Add Testing

Instruction:

Add unit tests for all endpoints using Jest and Supertest.

Outcome:

- Testing framework installed
- Test cases generated
- Errors fixed automatically

7. Custom Rules (AGENTS.md)

To enforce coding standards, a configuration file was added:

AGENTS.md

- Use MVC architecture
- Validate all inputs
- Use async/await
- Write tests for all endpoints

Instruction:

Refactor the code to follow AGENTS.md rules strictly.

Outcome:

- Code standardized
- Structure improved
- Consistency enforced

8. Advanced Feature Implementation

Authentication

Instruction:

Add JWT authentication and protect task routes.

Outcome:

- Authentication system implemented

- Middleware added
- Routes secured

9. Key Capabilities Demonstrated

9.1 Autonomous Development

OpenCode generated and modified code across multiple files without manual intervention.

9.2 Error Detection and Fixing

The agent identified runtime and logical errors and resolved them automatically.

9.3 Multi-step Execution

OpenCode executed complex workflows including:

- Code generation
- Dependency installation
- Testing
- Debugging

9.4 Rule-based Development

Using AGENTS.md, OpenCode followed predefined coding standards.

10. What Makes OpenCode Different

Traditional tools (e.g., autocomplete assistants) focus on:

- Suggesting code snippets
- Assisting within a single file

OpenCode introduces a new paradigm:

Traditional AI	Opencode
Suggests code	Execute Tasks
Works line-by-line	Works across entire projects
Passive assistant	Active agent
Single-step	Multi-step workflows

11. Core Capabilities

11.1 Autonomous Code Generation

- Converts natural language into working features
- Creates full project structures
- Implements backend, frontend, and integrations

11.2 Multi-Step Task Execution

OpenCode can:

- Analyze requirements
- Modify multiple files
- Run commands (install, build, test)
- Debug and fix issues automatically

11.3 Codebase Understanding

- Reads and analyzes existing repositories
- Identifies relevant files dynamically
- Maintains context across large projects

11.4 Agent-Based Architecture

OpenCode supports multiple agent roles:

- Build Agent → writes and modifies code
- Plan Agent → analyzes and proposes solutions
- Subagents → specialized roles (testing, debugging, etc.)

Enables safe and structured workflows

11.5 Tool Integration

Agents can use tools such as:

- Command execution (CLI)
- Web search and data retrieval
- User interaction (asking questions)

11.6 Multi-Model Support

- Supports 75+ AI providers
- Works with cloud and local models

Benefits:

- Flexibility in cost and performance
- No vendor lock-in
- Privacy control with local models

11.7 Custom Rules (AGENTS.md)

Developers can define project rules:

- Coding standards
- Folder structure
- Testing requirements

11.8 Plugin & Ecosystem Support

OpenCode is extensible through:

- Plugins
- External tools
- Workflow integrations

Examples:

- GitHub automation
- Monitoring tools
- CI/CD workflows

11.9 Multi-Interface Support

OpenCode can run in:

- Terminal (CLI-first)
- Web UI
- IDE integrations

11.10 GitHub Integration

Capabilities include:

- Issue analysis
- Bug fixing
- Pull request creation

- Branch management

12. Skills

Skills are reusable instruction modules that extend OpenCode's capabilities.

Unlike one-time prompts, skills provide structured workflows, best practices, and reusable logic that the agent can apply across tasks.

Structure

skills/

<skill-name>/

SKILL.md

examples.md (optional)

reference.md (optional)

scripts/ (optional)

How Skills Work

- OpenCode discovers skills from the repository or global config
- Skills are loaded on demand when relevant to a task
- The agent uses them as guidance for implementation

Benefits

- Consistent code quality
- Reusable patterns
- Faster development
- Team-wide standards

13. OpenCode Commands

OpenCode provides a command palette accessible by typing `/` in the terminal.

Session Management

- `/new` — Create a new session
- `/sessions` — Switch sessions
- `/fork` — Duplicate current session

Agent & Model Control

- `/agents` — Switch active agent
- `/models` — Change AI model
- `/connect` — Connect AI provider

Development Features

- `/skills` — View and use available skills
- `/init` — Generate AGENTS.md configuration
- `/editor` — Open file editor

Workflow Tools

- `/review` — Review changes (commit, branch, PR)
- `/export` — Export session transcript
- `/copy` — Copy session output

System & Utilities

- `/status` — View system status
- `/themes` — Change UI theme
- `/compact` — Reduce session size
- `/help` — Show help menu
- `/exit` — Exit application

14. Conclusion

OpenCode demonstrates significant potential as an AI-powered development agent. While not fully autonomous, it effectively reduces development effort by automating repetitive and multi-step tasks.

It is best suited for:

- Rapid prototyping
- Assisting developers
- Automating workflows

However, human oversight remains essential for:

- Code quality
- Security
- Production readiness